

LLctr 27: Complex Flow Control in C and ASM

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- The `break` and `continue` keywords for C and their implementation in ASM.
- The `switch` construct in C and implementations in ASM.

Demo project:

- Solutions of examples and exercises of the lecture.

C and ASM programming:

- The ADR and TBB instructions.

27.1 Introduction

In the previous lecture, we have discussed the `for`, `while`, and `do-while` loops. At the end of the lecture, we have an exercise about swapping pairs of chars in a string, as reproduced below.

Exercise 1. Write a function in C to swap the adjacent two chars in a string without calling existing C functions. Return the number of pairs that are swapped. Below are application examples:

- String “not” will be swapped to “ont” with the return being 1.
- String “note” will be swapped to “onet” with the return being 2.
- String “noted” will be swapped to “onetd” and the return being 2.

If you have worked on the problem carefully or checked the solution in the demo project code, you would find that the `while` loop is the best choice since we do not the number of iteration in advance so the `for` loop is not appropriate and we may not have a chance to swap a single pair so the `do-while` loop is not appropriate.

A version of the C code of the above problem is shown below.

```

1  int mp_swap_pairs_in_str_c4s(char *str) {
2      int pair_swapped = 0;    // # of pairs swapped
3      char ch1 = *str++;
4      char ch2 = *str;
5      while (ch1 && ch2) {
6          *str = ch1;
7          *(str - 1) = ch2;
8          ch1 = *++str;
9          ch2 = *++str;
10         pair_swapped++;
11     }
12     return pair_swapped;
13 }
```

To implement the compound test condition in the `while` loop in the above C using ASM, we just need to check if any of it is zero; if yes, we will branch out of the loop, as shown in the ASM code below.

```

1  @ int mp_swap_pairs_in_str_c4s_s(char *str);
2  @   Regs: *str -> r0;
3  @   Regs: ch1 -> r1, ch2 -> r2, pair_swapped -> r3,
4  mp_swap_pairs_in_str_c4s_s:
5      ldr    r3, =0
6      ldrb   r1, [r0], #1
7      ldrb   r2, [r0]
8  mp_swap_pairs_in_str_s_lp:
9      cbz    r1, mp_swap_pairs_in_str_s_end
10     cbz    r2, mp_swap_pairs_in_str_s_end
```

```
11      strb    r1, [r0]
12      strb    r2, [r0, #-1]
13      ldrb    r1, [r0, #1]!
14      ldrb    r2, [r0, #1]!
15      add     r3, #1
16      b       mp_swap_pairs_in_str_s_lp
17 mp_swap_pairs_in_str_s_end:
18      mov     r0, r3
19      bx      lr
```

One of the issues we face with the `while` loop, as illustrated in the code above, is the code repetition as we need to prepare for the condition test of the `while` loop before coming to the `while` loop. One technique we can use to overcome this issue is to use the `while (1)` loop with a `break` keyword, which **changes the flow in the loop by jumping out of it**, as illustrated below. Note that we have to wrap the `break` keyword in an `if` statement.

Another keyword used to change the flow of the loop is `continue`. We will discuss this as well.

The `break` keyword is also used in the `switch` construct to handle the *falling through* execution, which can be a designed feature.

The `switch` construct is implemented in ASM in a special way, and we will discuss the implement in detail.

Another related ASM instruction for flow control is the ADR instruction. We will discuss this instruction as well.

27.2 The `break` keyword for loop control and its implementation in ASM

As we have discussed before, one of the issues with the above implementations of the code for Exercise 1 of the previous lecture is the code repetition when building a `while` loop. Below, we will illustrate how to remove it with the `break` keyword.

A minor issue of the above code is that the repetition code blocks are not exactly the same. Before the loop, we have

```
1      char ch1 = *str++;
2      char ch2 = *str;
```

In the loop, we have

```
1     ch1 = *++str;  
2     ch2 = *++str;
```

This discrepancy in the repetition code was intentionally created to show that there are many possibilities to implement the same functionality. Now that we need to remove the code repetition, we need to rethink how to do it more systematically.

Example 1a. Write a function in C using a `while` loop with the `break` keyword to swap the adjacent two chars in a string without calling existing C functions. Return the number of pairs that are swapped. (Use the same application examples as shown above.)

Solution:

```
1  int mp_swap_pairs_in_str_break(char *str) {  
2      int pair_swapped = 0;  
3      while (1) {  
4          char ch1 = *str++;  
5          char ch2 = *str++;  
6          if (ch1 && ch2) {  
7              *(str - 1) = ch1;  
8              *(str - 2) = ch2;  
9              pair_swapped++;  
10         } else {  
11             break;  
12         }  
13     }  
14     return pair_swapped;  
15 }
```

:-

Note that this technique can be generalized to many different applications. Later in an Exercise, we will see a similar problem, which can be handled using a similar approach.

Now, let's see how the above code is implemented in ASM to have a better understanding of the `while (1)` construct and the `break` keyword.

Example 1b. Translate the above C code of Example 1a to ASM.

Solution:

```

1  @ int mp_swap_pairs_in_str_break_s(char *str);
2  @   Regs: *str -> r0;
3  @   Regs: ch1 -> r1, ch2 -> r2, pair_swapped -> r3,
4  mp_swap_pairs_in_str_break_s:
5      ldr    r3, =0
6  mp_swap_pairs_in_str_break_s_lp:
7      ldrb   r1, [r0], #1
8      cbz   r1, mp_swap_pairs_in_str_break_s_end
9      ldrb   r2, [r0], #1
10     cbz   r2, mp_swap_pairs_in_str_break_s_end
11         strb   r1, [r0, #-1]
12         strb   r2, [r0, #-2]
13         add    r3, #1
14     b     mp_swap_pairs_in_str_break_s_lp
15 mp_swap_pairs_in_str_break_s_end:
16     mov    r0, r3
17     bx     lr

```

:-

We can see that the `while` (1) construct and the `break` keyword are implemented without additional instructions.

27.3 The `continue` keyword for loop control and its implementation in ASM

While the `break` keyword changes the flow of any loop, a `for`, `while`, or `do-while` loop, by jumping out of it when a test condition is true, the `continue` keyword changes the flow of these loops in a different way, although it also requires a test condition to be true. Note that the `continue` keyword may require the change of the program structure, as discussed below.

- In the `for` loop, it jumps over the remaining code and go directly to the update expression to start the next pass of the loop.
- In the `while` or `do-while` loop, it jumps to the next `while` or `do-while` condition test. For this reason, we need to have the update of the loop control before the `continue` keyword.

Example 2a. Write a C function to count the number of chars in a string excluding the spaces.

Solution:

There are many approaches to implement this function. Here, we use the combination of the `break` and the `continue` keywords, as illustrated below.

```

1  int mp_count_non_space_chars_in_str(char *str) {
2      int num_chars = 0;
3      while (1) {
4          char ch = *str++;
5          if (ch == 0) {
6              break;
7          } else if (ch == ' ') {
8              continue;
9          }
10         num_chars++;
11     }
12     return num_chars;
13 }
```

-:-

Let's see how the `continue` keyword is implemented in ASM.

Example 2b. Translate the above C code of Example 2a to ASM.

Solution:

There are many approaches to implement this function. Here, we use the combination of the `break` and the `continue` keywords, as illustrated below.

```

1  @ int mp_count_non_space_chars_in_str_s(char *str);
2  @   Regs: *str -> r0;
3  @   Regs: ch -> r1, num_chars -> r2,
4  mp_count_non_space_chars_in_str_s:
5      ldr    r2, =0
6  mp_count_non_space_chars_in_str_s_lp:
7      ldrb   r1, [r0], #1
8      cbz   r1, mp_count_non_space_chars_in_str_s_end
9      cmp   r1, #32    @ ASCII code for ' ', the space
10     beq   mp_count_non_space_chars_in_str_s_lp
11     add   r2, #1
```

```
12      b      mp_count_non_space_chars_in_str_s_lp
13 mp_count_non_space_chars_in_str_s_end:
14      mov     r0, r2
15      bx      lr
```

:-

27.4 Implementation of the `switch` construct and its related `break` keyword in ASM

27.4.1 The `switch` construct in C

The generic format of the `switch` construct is shown below:

```
1  switch (expression) {
2  case constant1:
3      // statements
4      break;
5
6  case constant2:
7      // statements
8      break;
9
10     ...
11
12  default:
13      // default statements
14  }
```

Note that:

- The `break` keyword is used to prevent the execution of the code following it. If we remove the `break` keyword of the case `constant1`, the statements for case `constant2` will be executed as well. This can be used to our advantages when needed.
- The `default` keyword and its related statements inside the `switch` construct is optional. If they are used, no `break` is needed as this is the end of the construct already.

Let's see how this is used by using an example in C.

Example 3a. Write a C function to convert a capital case letter grade to a pass/fail grade. A grade of A, B, or C will be converted to P; D and F will be converted to F; all other letters will be reported as an error using letter E.

Solution:

The conversion can be done using an `if`-based construct. Here, we use a `switch` construct to do so, as shown below.

```
1 char mp_letter_grade_to_pass_or_fail(char letter_grade) {
2     char pass_fail_grade;
3     switch (letter_grade) {
4     case 'A':
5     case 'B':
6     case 'C':
7         pass_fail_grade = 'P'; // Pass
8         break;
9     case 'D':
10    case 'F':
11        pass_fail_grade = 'F'; // Fail
12        break;
13    default:
14        pass_fail_grade = 'E'; // Error
15    }
16    return pass_fail_grade;
17 }
```

:-

27.4.2 The `switch` construct in ASM using the naive approach

Example 3b. Translate the C function of Example 3a using a naive approach of implementing the `switch` construct.

Solution:

```
1 @ char mp_letter_grade_to_pass_or_fail_naive_s(char
   letter_grade)
2 @     Regs: letter_grade -> r0;
3 @     Regs: pass_fail_grade -> r0
4 mp_letter_grade_to_pass_or_fail_naive_s:
5     cmp     r0, #65     @ 'A'
```



```
6      beq      mp_letter_grade_to_pass_or_fail_naive_case_a
7      cmp      r0, #66      @ 'B'
8      beq      mp_letter_grade_to_pass_or_fail_naive_case_b
9      cmp      r0, #67      @ 'C'
10     beq      mp_letter_grade_to_pass_or_fail_naive_case_c
11
12     cmp      r0, #68      @ 'D'
13     beq      mp_letter_grade_to_pass_or_fail_naive_case_d
14     cmp      r0, #70      @ 'F'
15     beq      mp_letter_grade_to_pass_or_fail_naive_case_f
16
17     b        mp_letter_grade_to_pass_or_fail_naive_default
18
19 mp_letter_grade_to_pass_or_fail_naive_case_a:
20 mp_letter_grade_to_pass_or_fail_naive_case_b:
21 mp_letter_grade_to_pass_or_fail_naive_case_c:
22     ldr      r0, =80      @ 'P' for pass
23     b        mp_letter_grade_to_pass_or_fail_naive_end @ break
24
25 mp_letter_grade_to_pass_or_fail_naive_case_d:
26 mp_letter_grade_to_pass_or_fail_naive_case_f:
27     ldr      r0, =70      @ 'F' for fail
28     b        mp_letter_grade_to_pass_or_fail_naive_end @ break
29
30 mp_letter_grade_to_pass_or_fail_naive_default:
31     ldr      r0, =69      @ 'E' for error
32
33 mp_letter_grade_to_pass_or_fail_naive_end:
34     bx       lr
```

-:-

Note that the above code is relatively clean. Yet, the efficiency can be a bit low due to the repeated execution of comparison and branching. To handle this issue, ARM has provided an instruction to address this. This is the TBB, table branch byte, instruction, which is discussed below.

27.4.3 The `switch` construct in ASM using the TBB approach

27.4.3.1 Table checking with the TBB instruction

We have learned that to be able to find the handler of an specific interrupt quickly, ARM uses NVIC to host the entry point of each interrupt handler.

The TBB (table-based branch) instruction works in a similar approach. Unlike the vector table for NVIC which is 32-bit based, the table of the TBB instruction is 8-bit- or byte-based. Note that the Thumb-2 instruction is halfword aligned, which means the address of every instruction has to be an even number. So, to make good use of the byte numbers, the byte-table only contains the halved value of the **offset** address of each branch entry relative to a **special address**.

Assume we have a table named `mp_tbb_table`. Then, when we use the following TBB instruction,

```
1      TBB    [Rn, Rm]
```

we will branch to the address of $PC + 4 + \text{offset}$.

The value of `offset` is obtained by

```
1      mp_tbb_table[Rm] * 2
```

where the table `mp_tbb_table` is saved at the address pointed by `Rn`.

Note that the value of `Rm` should be between 0 and $n - 1$ if we use a table of n elements.

We can use an example to understand the above instruction after we discuss another new instruction.

27.4.3.2 Relative branching with the ADR instruction

The ADR (taking the relative address) instruction is used to load a PC-relative address, which is used in position-independent code.

The syntax of the ADR instruction is shown below.

```
1      ADR{cond} Rd, label
```

Here, `cond` is the CCS we have discussed before, and `label` is a PC-relative expression, and will

load the value of the address of `lable` to `Rd`. This will be easier to understand with an example, as shown below.

27.4.3.3 Examples

Example 3c. Translate the C function of Example 3a using the TBB-based `switch` construct.

Solution:

```

1  @ char mp_letter_grade_to_pass_or_fail_tbb_s(char letter_grade
   )
2  @   Regs: letter_grade -> r0;
3  @   Regs: pass_fail_grade -> r0
4  mp_letter_grade_to_pass_or_fail_tbb_s:
5      sub     r0, #65      @ minus 'A' to => choices: 0, 1,...
6      cmp     r0, #0
7      blt     mp_tbb_default
8      cmp     r0, #6
9      bge     mp_tbb_default
10     adr     r1, mp_tbb_table
11     tbb     [r1, r0]
12
13 mp_tbb_case_a:
14 mp_tbb_case_b:
15 mp_tbb_case_c:
16     ldr     r0, =80      @ 'P' for pass
17     b       mp_letter_grade_to_pass_or_fail_tbb_end @ break
18
19 mp_tbb_case_d:
20 mp_tbb_case_f:
21     ldr     r0, =70      @ 'F' for fail
22     b       mp_letter_grade_to_pass_or_fail_tbb_end @ break
23
24 mp_tbb_default:
25     ldr     r0, =69      @ 'E' for error
26
27 mp_letter_grade_to_pass_or_fail_tbb_end:
28     bx      lr
29
30 mp_tbb_table:
31     .byte   0              @ r0 = 0, 'A'
32     .byte   (mp_tbb_case_b - mp_tbb_case_a) / 2 @ r0 = 1, 'B'

```

```
33      .byte    (mp_tbb_case_c - mp_tbb_case_a) / 2 @ r0 = 2, 'C'
34      .byte    (mp_tbb_case_d - mp_tbb_case_a) / 2 @ r0 = 3, 'D'
35      .byte    (mp_tbb_default - mp_tbb_case_a) / 2 @ r0 = 4, 'E'
36      .byte    (mp_tbb_case_f - mp_tbb_case_a) / 2 @ r0 = 5, 'F'
```

The above code deserves some notes:

- Line 5. The expected letters are from A to F, with E being excluded. So, when subtracting 65, which is the ASCII code for A, we will be able to have the value of R0 to be between 0 and 5, inclusive, excluding 4. This is needed to check the table at `mp_tbb_table`.
- Lines 6 to 9. Use to handle unexpected cases, such letter G.
- Line 10. Load the value of `mp_tbb_table` to R1 so that it will be used as the base for the table used by the TBB instruction.
- Line 11. Branch to the respective entry points of the different cases using the value checked from the `mp_tbb_table`.
- Line 13. The label for the entry point corresponding to the value of R0 being 0. This is indexed by the data saved in Line 31.
- Line 14. The label for the entry point corresponding to the value of R0 being 1. This is indexed by the data saved in Line 32.
- Line 17. This is the implementation of the `break` keyword.
- Line 31. The value 0 at this line means that the execution will be in the next instruction immediately following the TBB instruction on Line 11.
- Line 32. This value is used to branch to `mp_tbb_case_b` by the TBB instruction.
- Line 35. This value is used to branch to `mp_tbb_default` by the TBB instruction. This is needed as we have a hole between the expected letters D and F.

27.4.3.4 The TBH instruction

Note that TBB has a related instruction, TBH, table branch halfword. The difference between the TBH and TBB instructions is that the former uses a table of halfword, which implement bigger `switch` constructs that what TBB can do.

27.5 Class exercises

Exercise 1a. Write a function in C to swap the first and third letters of each trio in a string without calling existing C functions. Return the number of trio that are swapped. Below are application examples:

- String “not” will be swapped to “ton” with the return being 1.
- String “note” will be swapped to “tone” with the return being 1.
- String “noted” will be swapped to “toned” and the return being 1.
- String “denote” will be swapped to “nedeto” and the return being 2.

Exercise 1b. Translate the C function of Exercise 1a into ASM.

See the demo project for the above two functions.